

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 Modules

02 Examples

03

04

05

Modules

- We have seen how a particular piece of code is reused in functions
- Modules go a step ahead
- Modules allow you to reuse one or more functions in your program

Modules

- Module is a file with .py extension that has definitions of all functions and variables that you would like to use even in other programs
- The program in which you want to use functions or variables defined in the module will simply import that particular module (or .py file)

Modules

- Modules are pre-written pieces of code that are used to perform common tasks.
- Basic way to use a module is to add `import module_name` as the first line of your program
- Then writing `module_name.var` to access functions & values

Example Modules

```
import sys      #System module is imported using it
print("\n PythonPath= \n", sys.path)
```

- When the `import sys` is executed, Python looks for the `sys.py` module in the directories in its `sys.path` variable
- If the file is found, then the statements in the module is executed

Modules

- A module imported in a program must be located and loaded into memory before it can be used
- Python 1st searches for the modules in the current working directory
- If not found, it then looks for the module in the Python path environment
- If not found, then looks in Python installation-specific path (like C:\Python39\Lib)

Modules

- If not found, then Import error is generated

Example Modules

```
import math      # Math module is imported using it
Py_val = math.pi
print(math.pi)
3.141592653589793
```

from... import statement

- A module may contain definition for many variables and functions
- When you import a module, you can use any variable or function defined in that module
- If you want to use only a selected variables or functions, then you can use: `from... import statement`

from... import statement

```
from math import pi  
print(pi)  
3.141592653589793
```

from... import statement

- To import more than one item from a module, use a comma separated list.
- For example, to import the value of pi and sqrt() from the math module, you can write,

```
from math import pi, sqrt
```

from... import statement

- You can also import a module with a different name using 'as' keyword
- This is particularly more important when a module either has a long or confusing name

```
from math import sqrt as square_root
```

```
Print(square_root(16))
```

```
4.0
```

Making your own Modules

- you can easily create as many modules as you want
- In fact, you have already been doing that
- Every Python program is a module, i.e. every file that you save as .py extension is a module

Making your own Modules

- First write the program in a file and save as MyModule.py (You can give any other name if you want)

#Making My own Module, for factorial Calculation

```
def fact(n):  
    if(n==1 or n==0):  
        return 1  
    else:  
        return n*fact(n-1)
```

Making your own Modules

- Then, open another file (main.py) and write the lines as follows:

```
import MyModule  
n = int(input("Enter a number: "))  
Fact_val = MyModule.fact(n)  
print("Factorial of",n,"is",Fact_val)
```

Standard Library Modules

- Python supports three types of modules:
 - Those written by the programmer
 - Those that are installed from external sources, and
 - Those that are pre-installed in Python (are called standard library)

Standard Library Modules

- Some important modules in the standard library are:
 - string
 - re
 - datetime
 - math
 - random
 - os
- You can use them for various tasks

Standard Library Modules Example

```
import calendar  
print(calendar.month(2023,12))
```

```
    December 2023  
Mo Tu We Th Fr Sa Su  
      1  2  3  
 4  5  6  7  8  9 10  
11 12 13 14 15 16 17  
18 19 20 21 22 23 24  
25 26 27 28 29 30 31
```

The dir() Function

- **dir()** is a built-in function that lists the identifiers defined in a module.
- These identifiers may include functions, classes and variables.
- If no name is specified, the dir() will return the list of names defined in the current module.

The dir() Function

```
def print_var(x):  
    print(x)  
x = 10  
print_var(x)  
print(dir())
```

OUTPUT

```
10  
['__builtins__', '__doc__', '__file__', '__name__', '__package__', 'print_var', 'x']
```

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025